PROTOCOLO EXPERIMENTAL: CICLO DE VIDA E GESTÃO DE ESTADO EM ANDROID NATIVO

Tema 2: Ciclo de Vida e Gestão de Estado (Divisão 50/50 com o Grupo 4)
Grupo 3: Letizia, Júlia, Bruno
Aplicação de Teste: LifeStateLab (Kotlin e módulo de equivalência Java)
Repositório Git: https://github.com/BrSalv2006/PDM-Ciclo-de-vida-e-gestao-do-estado

1. Enquadramento Teórico

1.1. O Modelo Orientado a Eventos do Android

Ao contrário de ecossistemas convencionais orientados a um método estático `main()`, os componentes no sistema operativo Android são instanciados, geridos e destruídos pelo próprio sistema. As aplicações reagem a transições de estado orquestradas pelo ActivityManagerService em resposta a ações do utilizador (mudanças de foco, receção de chamadas) ou a constrangimentos físicos de hardware (pressão de memória RAM).

O ciclo de vida canónico da Activity rege-se por cinco fases fundamentais:

- `onCreate()`: Ponto de entrada inicial onde se realiza o carregamento e a instanciação de layouts através de View Binding, bem como a vinculação de estruturas essenciais.
- `onStart()`: Momento em que a janela se torna visível ao utilizador, entrando na hierarquia do ecrã.
- `onResume()`: A `Activity` assume o primeiro plano (foreground) e adquire o foco de interação ativa.
- `onPause()` / `onStop()`: Perda de foco e de visibilidade. Constitui o ponto obrigatório para suspender animações, interromper transmissões e libertar sensores de hardware para evitar a degradação da bateria.
- `onDestroy()`: Desalocação definitiva da instância na memória do processo.

1.2. Desacoplamento da Interface Gráfica nos Fragments

Nas arquiteturas modernas com Single-Activity, a interface modulariza-se em `Fragments`. A particularidade arquitetural mais crítica reside no facto de o ciclo de vida da interface gráfica (`View`) do Fragment ser mais efémero do que o ciclo de vida da instância do próprio Fragment.

Durante a navegação na pilha de recuo (back stack), o Fragment permanece alocado na memória, mas a sua View é destruída através de onDestroyView(). A omissão da limpeza de referências (através de `_binding = null`) retém toda a hierarquia de Views na memória heap, originando fugas de memória (Memory Leaks) irreversíveis até ao término do `Fragment`.

1.3. Alterações de Configuração e Persistência de Estado

Ao ocorrer uma alteração de configuração (Configuration Change), como a rotação física do dispositivo entre Portrait e Landscape, o Android destrói a Activity e recria-a do zero para permitir a recomposição de recursos otimizados (como o qualificador `res/layout-land/`). Isto acarreta a perda imediata de quaisquer variáveis voláteis mantidas na memória da Activity.

A arquitetura do Android estrutura a retenção de dados em três mecanismos complementares:

1. `onSaveInstanceState(Bundle)`: Serialização de pares chave-valor primitivos antes da destruição. Opera sobre o mecanismo de IPC (Binder) do SO, possuindo um limite global restrito de aproximadamente 1 MB para todas as transações concorrentes do processo. O armazenamento indevido de listas ou imagens origina a exceção terminal `TransactionTooLargeException`.
2. `ViewModel`: Componente retido em memória RAM pelo `ViewModelStoreOwner` durante a destruição e recriação da `Activity`. Elimina a necessidade de repetição de pedidos de rede e de reprocessamento de coleções pesadas.
3. `SavedStateHandle`: Mecanismo de salvaguarda injetado no `ViewModel` para mitigar a Morte de Processo (System-initiated Process Death). Quando o sistema operativo encerra o processo por pressão de memória RAM, o `ViewModel` em RAM é purgado a zero, mas o `SavedStateHandle` persiste identificadores e dados chave nos serviços centrais do sistema, restaurando-os no arranque subsequente.

1.4. Reatividade na Interface: LiveData vs. StateFlow

A propagação reativa do estado para a interface gráfica segue o padrão Observer:

- `LiveData` (Java / Android Jetpack): Mecanismo nativamente consciente do ciclo de vida (lifecycle-aware). Suspende emissões enquanto o hospedeiro estiver em segundo plano (abaixo de `STARTED`), evitando erros de contexto visual e `NullPointerExceptions`.
- `StateFlow` (Kotlin Coroutines): Fluxo de estado determinístico, moderno e agnóstico ao ecossistema Android. Exige obrigatoriamente um valor inicial e disponibiliza um ecossistema completo de operadores funcionais. Contudo, por não possuir perceção intrínseca do ciclo de vida da interface gráfica, a sua recolha segura num `Fragment` requer estritamente o uso da API `repeatOnLifecycle(Lifecycle.State.STARTED)`. A recolha com `lifecycleScope.launch` simples não interrompe a coroutine em background, consumindo ciclos de CPU e bateria indevidamente.

2. Protocolo Experimental (Reprodutibilidade)

2.1. Objetivos do Ensaio

1. Demonstrar quantitativamente a taxa de persistência de dados perante a rotação física do ecrã (variável volátil vs. Bundle vs. ViewModel).

2. Monitorizar a sequência de execução e a simetria de callbacks entre `Activity` e `Fragment` em tempo real através de um monitor de diagnóstico na interface.
3. Validar empiricamente a resiliência à morte de processo por pressão de memória através de `SavedStateHandle`.
4. Demonstrar a reatividade concorrente entre `LiveData` e `StateFlow` com suspensão segura em segundo plano.

2.2. Configuração do Ambiente de Teste

- **Dispositivo / Emulador:** Android Virtual Device ou Dispositivo Físico com Android 10 (API 29) ou superior.
- **Ferramenta de Desenvolvimento:** Android Studio (Hedgehog / Koala ou superior).
- **Stack Tecnológica:** Kotlin 1.9+ com módulo comparativo em Java 11/17 (`MainViewModelJava.java`), Gradle 8.x, View Binding ativado.
- **Dependências Jetpack:** `lifecycle-viewmodel-ktx:2.8.7`, `lifecycle-livedata-ktx:2.8.7`, `lifecycle-runtime-ktx:2.8.7`, `fragment-ktx:1.8.5`.

2.3. Procedimento Experimental Passo a Passo

Ensaio A: Alteração de Configuração (Rotação de Ecrã)

1. Abrir a aplicação **LifeStateLab** com orientação vertical (Portrait).
2. Pressionar quatro vezes consecutivas o botão **"+ Contadores"**, elevando os indicadores ao valor 4.
3. Rodar o dispositivo para a orientação horizontal (Landscape).
4. Registar a adaptação ao layout `res/layout-land/` em duas colunas funcionais.
5. Observar o valor dos contadores e a sequência de callbacks registada no monitor.

Ensaio B: Ciclo de Vida do Fragment e Desacoplamento da View

1. Na aplicação em execução, pressionar **"Abrir Fragment"**.
2. Registar os eventos no monitor: `[FRG] onAttach()`, `[FRG] onCreate()`, `[FRG] onCreateView()`, `[FRG] onViewCreated()`, `[FRG] onStart()`, `[FRG] onResume()`.
3. Pressionar **"Fechar Fragment"** (ou a tecla física Back).
4. Registar o encerramento da View: `[FRG] onPause()`, `[FRG] onStop()`, `[FRG] onDestroyView()`, `[FRG] onDestroy()`, `[FRG] onDetach()`, confirmando a anulação `_binding = null`.

Ensaio C: Morte de Processo (Process Death)

1. Abrir o Fragment e digitar na caixa de texto a cadeia: `"PDM 2026 – Grupo 3"`.
2. Incrementar os contadores até ao valor 4.
3. Nas Opções de Programador do Android, ativar **"Não manter atividades"** (Don't keep activities).

4. Premir o botão Home para enviar a aplicação para background (provocando o descarte imediato da memória do processo).
5. Retomar a aplicação através do painel de aplicações recentes.
6. Registar os valores do ViewModel e o texto recuperado via `SavedStateHandle`.

Ensaio D: Reatividade e Atualização de Estado

1. Com o Fragment visível, clicar 3 vezes no botão **"+ Flow"** e 5 vezes no botão **"+ LiveData"**.
2. Confirmar a reflexão síncrona dos valores na interface.
3. Minimizar a app e verificar no Logcat a suspensão da coroutine no ponto de transição do ciclo de vida.

3. Tabela de Métricas e Resultados Experimentais

Parâmetro / Componente	Estado Inicial	Evento Aplicado	Resultado Observado	Avaliação Arquitetural
Variável Simples (Memória ...	4	Rotação (Portrait → Landsc...	0 (Reset)	**0% Retenção:** Perda por destruição da instância
Bundle (onSaveInstanceSta...	4	Rotação (Portrait → Landsc...	4 (Preservado)	**100% Retenção:** Restaurado com sucesso via IPC
ViewModel (RAM do Proce...	4	Rotação (Portrait → Landsc...	4 (Preservado)	**100% Retenção:** Sobrevivência na memória RAM
ViewModel (RAM do Proce...	4	Morte de Processo (Proces...	0 (Reset)	**0% Retenção:** Destruição total da memória do processo
SavedStateHandle (Texto d...	"PDM 2026 – Grupo 3"	Morte de Processo (Proces...	"PDM 2026 – Grupo 3"	**100% Retenção:** Restauro transparente a partir do SO
View Binding do Fragment	Ativo	Fecho do Fragment	`_binding = null`	**Desalocação completa da View:** Zero Memory Leaks
StateFlow / Coroutines	Ativo	App em Background	Suspenso (STARTED)	**Otimização:** Zero desperdício de CPU/bateria

4. Conclusões e Recomendações

Os resultados empíricos validam integralmente as diretrizes arquiteturais recomendadas pela Google:

1. Variáveis locais e estados provisórios de interface não devem ser utilizados para persistência além de interações imediatas.

2. O `ViewModel` representa a via ideal para preservar o estado perante rotações, aliviando o canal de IPC.
3. O `SavedStateHandle` preenche a lacuna imperativa da morte forçada de processos pelo sistema operativo.
4. O consumo de fluxos reativos `StateFlow` exige imperativamente a disciplina de ciclo de vida com `repeatOnLifecycle`, nivelando a sua robustez face ao `LiveData` histórico.